

Building a New Project

An Engineering & Design Onboarding Guide Template

Engineering

2026

Contents

Welcome	4
Chapter 1 — The Big Picture	5
What this structure buys you	5
A workspace layout worth starting from	5
The dependency chain	6
Chapter 2 — Monorepos, pnpm, and Turborepo	7
Why this tooling pairing	7
The commands worth standardizing on	7
Chapter 3 — Design Tokens: the Language of the Design System	9
What a design token is	9
A token package worth building	9
Following one value end to end	10
Chapter 4 — Atomic Design: Organizing the Component Library	13
The methodology, briefly	13
A five-tier structure worth adopting	13
The file shape every component should follow	14
Chapter 5 — Styling: styled-components and Transient Props	16
The problem transient props solve	16
The pattern worth using consistently	16
A shared spacing-prop mixin	16
Chapter 6 — Light and Dark Mode	18
The mechanism worth copying	18
Chapter 7 — Implementing the UI Kit on the Frontend	20
Consuming the design system in a real app	20
Separating API requests from presentational components	20
A folder structure worth starting from	21
Chapter 8 — Backend Fundamentals	23
Prisma: a type-safe data layer	23
Rate limiting	23
Auth guards	24
Authentication and authorization: which to use when	24
Encryption and hashing	25
What to avoid for a fast, secure API	26
Chapter 9 — Quality Tooling	27
ESLint: one shared flat config	27
TypeScript: a small set of shared configs	27
Testing: pick per package, not per project	27

Chapter 10 — Storybook & Component-Driven Development	29
Why build components in isolation	29
A Storybook setup worth adopting	29
Chapter 11 — Continuous Integration	30
Chapter 12 — Cheat Sheet	31
Commands	31
Where things should live	31
Chapter 13 — Where to Start, By Role	32
If you're a junior developer	32
If you're a senior developer	32
If you're a product designer	32
Appendix — Further Reading	33

Welcome

This is a template onboarding guide for a new engineering team: a starting point you adapt with your own project's real names, ports, and file paths once they exist. It's meant for anyone joining a project built as a **monorepo with a shared design system** — junior developers, senior engineers touching the build pipeline for the first time, and product designers handing off files to be implemented.

Most real-world products end up needing the same handful of systems: a monorepo that ties a frontend, a backend, and shared packages together; a two-layer design token system that keeps color, spacing, and type consistent; an Atomic Design component library built on those tokens; a clear line between data-fetching and presentation on the frontend; and a backend that gets authentication, rate limiting, and data access right from day one. This guide walks through each of those systems once, in the order you'd actually build them, so a new project doesn't have to rediscover the same decisions from scratch.

Each chapter explains the general concept first — for anyone new to that idea — and then shows a concrete, adaptable implementation with real code shapes you can copy into a fresh repository. You don't need to read it front to back. Skim the table of contents, jump to what's relevant to your work this week, and come back to the rest as you need it.

Throughout the guide you'll see a few kinds of boxes. A blue box marked **"For designers"**, **"For junior devs"**, or **"For senior devs"** calls out something specific to that audience. A teal **key fact** box highlights a rule worth remembering. A dark **"Avoid"** box flags a common mistake.

The one-sentence version of this whole guide: raw design values live in a tokens package, get turned into CSS variables and Storybook-documented components in a UI package, get consumed by a frontend app that keeps data-fetching separate from presentation, and are backed by an API that authenticates, authorizes, validates, and rate-limits every request before it ever reaches the database.

Chapter 1 --- The Big Picture

What this structure buys you

A monorepo is a single repository holding multiple, independently-versionable packages, instead of splitting a design system, a frontend, and a backend across separate repositories. The appeal: a designer changing a token value and an engineer consuming that token in a component can make and review both changes in one pull request, with the compiler catching any mismatch immediately, instead of coordinating a release across three repos.

A workspace layout worth starting from

A **pnpm workspace** is a good default: `pnpm-workspace.yaml` declares that anything under `apps/` or `packages/` is a workspace member, and every member gets its own `package.json` and scripts while sharing one install and one lockfile at the root.

A minimal, proven split is two `apps/` (things that get deployed — a web frontend and an API service) and a handful of `packages/` (things that get consumed by other packages — a component library, a design-token library, and shared tooling configs).

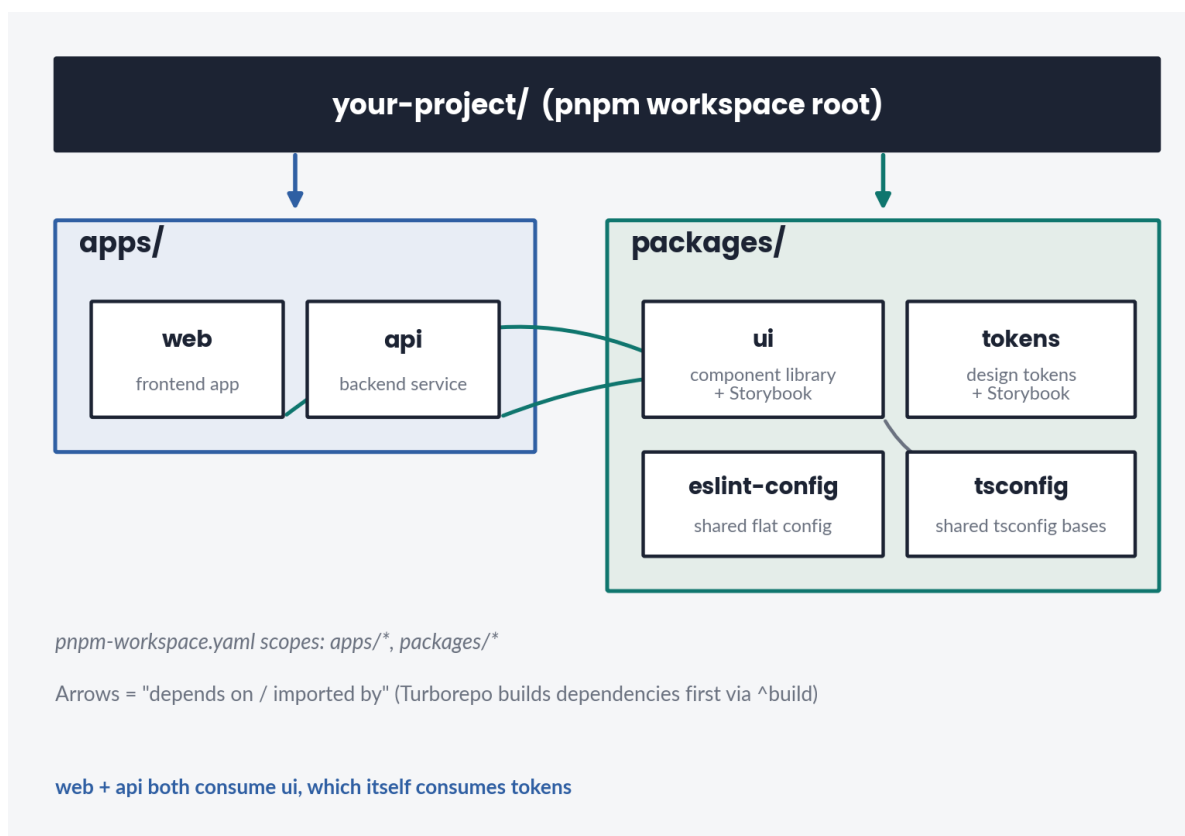


Figure 1: The monorepo workspace: apps consume packages, and packages consume each other

For junior devs

If you've only worked in single-repo projects before, the mental model to hold onto is:

packages/ hold libraries, apps/ hold products. Most early tasks are either adding a component to the UI package or wiring an existing component into a frontend page — rarely inventing something from nothing.

The dependency chain

apps/web and apps/api both depend on the ui package. ui in turn depends on the tokens package. Nothing depends on the two apps — they're leaves in the dependency tree, which is exactly what you'd expect from deployable applications. Every package additionally pulls in shared eslint-config and tsconfig packages as dev-time tooling dependencies.

Chapter 2 --- Monorepos, pnpm, and Turborepo

Why this tooling pairing

pnpm is the package manager. Its distinguishing feature versus npm or Yarn Classic is a content-addressable store: every version of every package is stored once on disk, and every project's `node_modules` is built from links into that store, which keeps installs fast and disk-cheap. Its more important property here is **workspaces**: pnpm understands that a package referenced as `workspace:*` should resolve to local source rather than a published npm version, so a change to a shared package is instantly visible to anything that imports it, with no publish step.

Turborepo decides which scripts run, in what order, and whether they need to run at all. It reads a `turbo.json` at the repo root, which defines a **task graph**: a description of which tasks depend on which other tasks, across every workspace package at once. The pattern worth copying: `build`, `lint`, `check-types`, and `test` should all declare `dependsOn: ["^build"]` — the caret means “the build task of every package this package depends on.” In practice that means running tests for the frontend app doesn't just run its own tests; Turborepo first walks the dependency graph, rebuilds the tokens and UI packages if their source changed, and only then runs the tests, so the app is always tested against fresh output rather than stale or raw source.

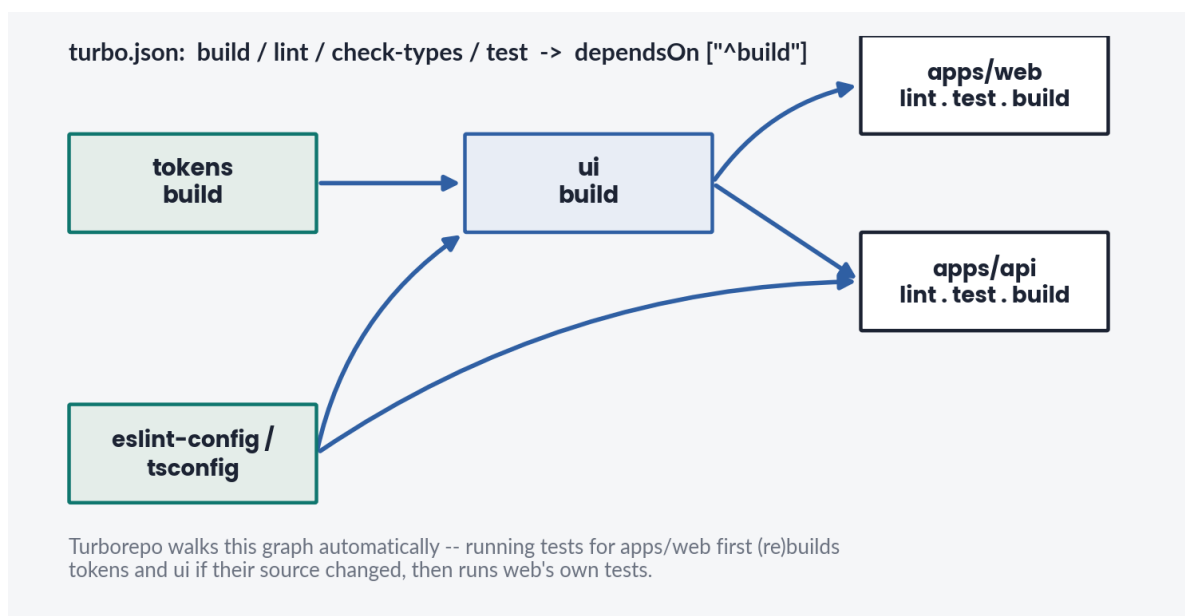


Figure 2: Turborepo's task graph: build dependencies before testing or building what depends on them

Turborepo also **caches** task outputs. If nothing that could affect a package's build has changed, Turborepo replays the cached result instead of re-running the task — which is why CI stays fast even as a repo grows, and part of why CI should cache the `.turbo` directory between runs.

The commands worth standardizing on

```
pnpm install          # one install for the whole workspace
pnpm dev              # long-running dev servers (not cached)
```

```
pnpm build          # turbo run build, dependency-ordered
pnpm lint           # eslint --max-warnings 0, per package
pnpm check-types    # tsc --noEmit, per package
pnpm test           # per-package test runner
pnpm format:check   # prettier --check . (not turbo'd – runs once, repo-wide)
pnpm storybook      # turbo run storybook, all packages that have one
```

Scope any of these to a single package with `--filter`, e.g. `pnpm --filter @yourorg/ui test` — useful when iterating on one package without waiting on the whole graph.

For designers

You will likely never type most of these commands directly. The one worth knowing is `pnpm storybook`, which starts a local server showing every component in isolation — the fastest way to browse what already exists before asking an engineer to build something new. See Chapter 10.

Chapter 3 --- Design Tokens: the Language of the Design System

What a design token is

A design token is a named, stored design decision — a color, a spacing value, a font size — kept as data instead of hard-coded inside components. Naming a hex code as a token rather than writing it directly into a Button's CSS means the *name* can be reused everywhere that color is needed, and the *value* behind that name can change in one place without hunting through every component that uses it.

Mature token systems split tokens into two layers, and it's worth building a new project this way from the start.

Primitive tokens are raw, context-free values — the complete palette of everything the system could possibly use. A primitive doesn't know what it's *for*; a token like `brand-blue-500` is just a hex code.

Semantic tokens are purpose-driven names that *reference* primitives. `color-accent` doesn't contain a hex code itself — it points at `brand-blue-500` — and it's the semantic name, not the primitive, that components actually use. This indirection is what makes theming possible: swap what `color-accent` points to, and every component using it updates automatically, with zero component code changed.

For designers

If you're used to Figma variables, this maps almost exactly onto Figma's own primitive-vs-semantic (or "raw" vs "alias") variable modes. A token named for what it *is* (`blue-500`) is a primitive; a token named for what it's *used for* (`color-accent`, `color-danger`) is semantic. When you hand off a design, referencing the semantic name in your notes ("this button uses `color-accent`") is far more durable than referencing a hex code.

A token package worth building

A dedicated packages/tokens library should own the actual values: files like `defaults/colors.ts`, `spacing.ts`, `typography.ts`, and `fonts.ts` each exporting a flat object of primitive tokens, for example:

```
// packages/tokens/src/defaults/colors.ts
'brand-blue-500': '#2f5fa3',
// ...and semantic aliases, in the same file:
'color-text-primary': 'var(--brand-ink-900)',
'color-accent': 'var(--brand-blue-500)',
'color-accent-hover': 'color-mix(in oklch, var(--brand-blue-500) 85%, black)',
```

Dark mode should be defined right alongside light mode, in a second export (`darkColorTokens`) that reuses the *exact same semantic key names* — `color-bg`, `color-accent`, and so on — but points each one at a different primitive. That reuse of names across light and dark is the entire foundation the dark-mode mechanism in Chapter 6 depends on.

`spacing.ts` should define a rhythm scale (`space-0` through `space-32`, say 2px to 128px), radii, border widths, shadows — including a shadow token that itself references a color token (`shadow-focus: 0 0 0 3px var(--color-focus-ring)`) — and motion tokens like `ease-standard` and

duration-*. typography.ts should define font families and weights, plus composite tokens that bundle weight, size, line-height, and family into a single CSS font shorthand.

A second layer belongs in a createSemanticTokens.ts: a pure function that takes primitive references in and derives *component-anatomy* tokens out — things like inset-*/stack-*/inline-* (spacing rhythm), control-height-*/control-pad-x/control-radius (the anatomy of a form control, shared by Button, Input, Select), surface-radius/surface-shadow-* (card anatomy), and focus-ring/focus-ring-offset.

A mergeTokens(defaults, overrides) helper is worth keeping deliberately trivial — a shallow { ...defaults, ...overrides } — because it's the escape hatch that makes future rebranding cheap: a client project only needs to override the handful of tokens that change.

The packages/ui package should consume tokens through one thin file per token type, each importing the defaults, passing them through mergeTokens, and serializing the result into a CSS string. Colors are the one type that emits two CSS blocks from the same function:

```
// packages/ui/src/theme/tokens/colors.ts
export const colorsCss =
  tokensToCss(mergeTokens(lightColorTokens)) +
  tokensToCss(mergeTokens(darkColorTokens), { selector: "[data-theme='dark']" });
```

A base.ts isn't tokens at all — it's a plain CSS reset (box-sizing, default background/text color pulled from color tokens, a focus-visible ring built from shadow-focus) that establishes sane defaults on raw HTML elements. A barrel file should concatenate all of these CSS strings — fonts, colors, spacing, typography, semantic, base — into one styled-components createGlobalStyle block, mounted once by a ThemeProvider at the root of the app.

Finally, a hand-maintained typed mirror (theme.ts) of the same values as a plain JS object exists purely so props.theme.color.accent autocompletes in an editor inside a styled-components template literal — it should never become a second source of truth. props.theme.color.accent and hardcoding var(--color-accent) resolve to the identical string, because both point at the same CSS custom property.

Following one value end to end

The clearest way to internalize this system is to trace a single value all the way from its definition to a rendered pixel:

1. **Primitive**, in tokens/src/defaults/colors.ts: 'brand-blue-500': '#2f5fa3'.
2. **Semantic alias**, same file: 'color-accent': 'var(--brand-blue-500)'.
3. **Serialized to CSS**, in ui/theme/tokens/colors.ts: written into :root { --color-accent: var(--brand-blue-500); }, and under the dark palette into [data-theme='dark'] { --color-accent: <dark value>; }.
4. **Injected once**, via GlobalStyle, mounted by ThemeProvider.
5. **Consumed directly**, in Button.styles.ts: background: var(--color-accent); — not props.theme.color.accent, though it resolves identically.

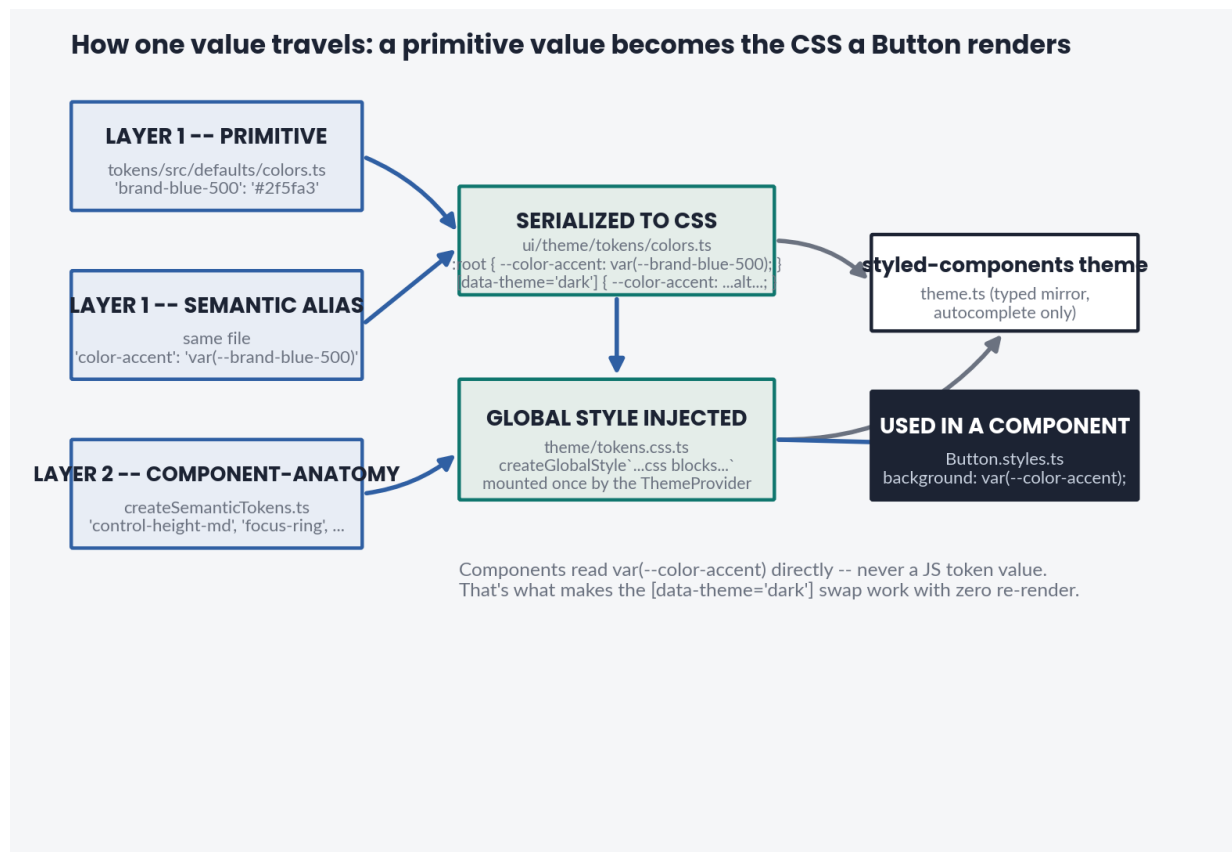


Figure 3: One value's journey: a primitive value becomes the background of a Button

Components should read CSS custom properties (`var(--color-accent)`) directly. They should never read a JavaScript token value at render time. This one decision is what makes dark mode a zero-JavaScript, zero-re-render CSS cascade — covered fully in Chapter 6.

For senior devs

Skip a runtime rebrand prop on `ThemeProvider` until you actually need one — token composition can stay build-time only, resolved once when `theme/tokens/*.ts` calls `mergeTokens()`. `mergeTokens` already exists for exactly the moment you need runtime brand-switching instead of compile-time; it's just an unused seam until then.

Chapter 4 --- Atomic Design: Organizing the Component Library

The methodology, briefly

Atomic Design is a way of organizing UI components by increasing complexity, borrowed from chemistry's own hierarchy: atoms combine into molecules, molecules combine into organisms. Brad Frost formalized it as five tiers — atoms, molecules, organisms, templates, and pages — and its real value isn't the taxonomy itself but the **shared vocabulary** it gives a team. When a designer or engineer says "this should be an atom," everyone in the room understands roughly how small, how reusable, and how opinion-free that piece should be.

The rule that makes the hierarchy actually hold together in code, not just in theory, is that **each tier is composed only from the tier below it** — an atom never reaches sideways into another atom's internals, and a molecule never skips straight to raw CSS when an atom already exists for the job.

A five-tier structure worth adopting

It's worth adding one tier underneath Brad Frost's original three: **primitives** — the truly irreducible layout building block beneath atoms. Above organisms, instead of generic "templates" and "pages," a single product usually only needs concrete **app sections**.

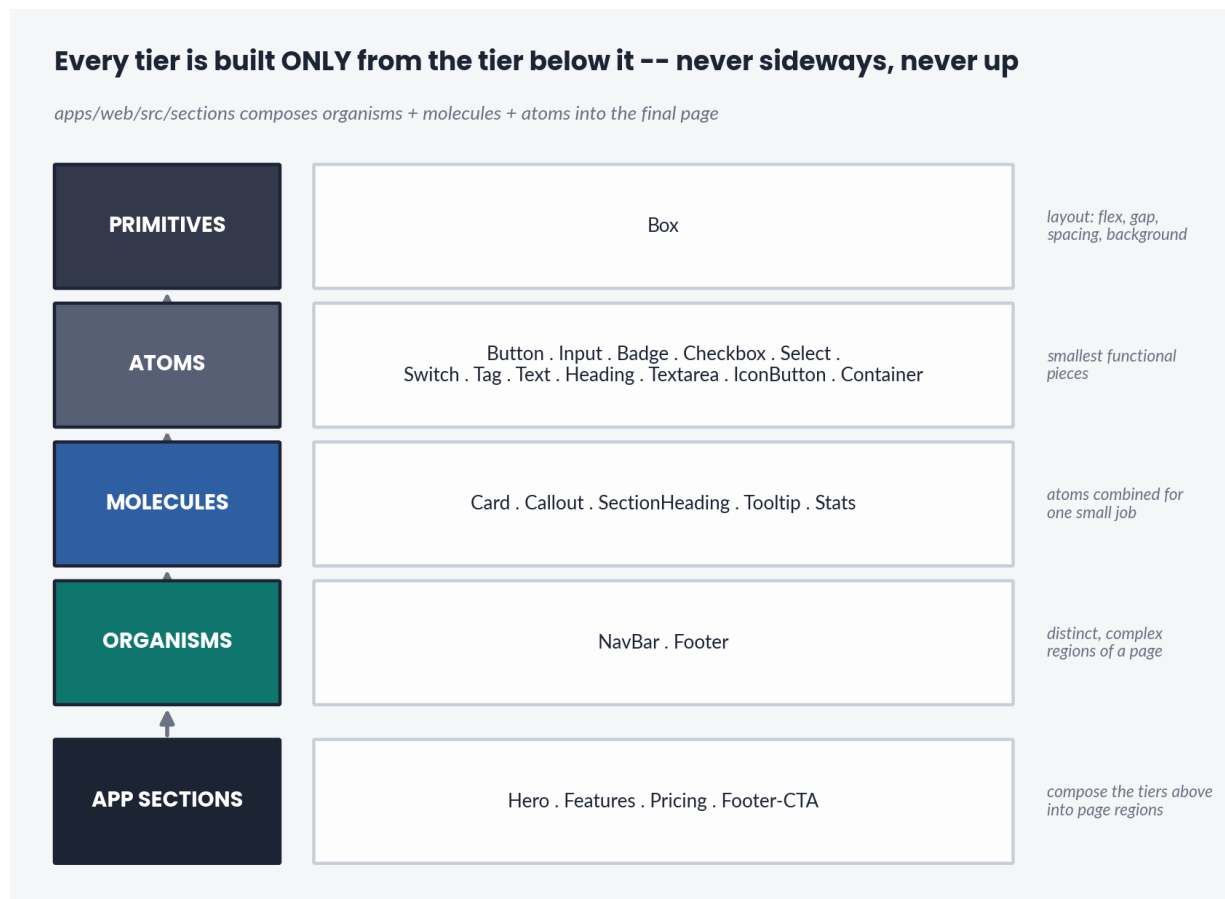


Figure 4: Every tier should be built only from the tier directly below it

Primitives (packages/ui/src/primitives/): start with just Box — the load-bearing layout primitive

that every other layout-oriented component should build on rather than styling a bare `div`. It owns `flex/gap/alignment` props, background variants, and a shared spacing-prop mixin (below).

Atoms (`src/atoms/`): the smallest pieces that do something functional on their own — Button, Input, Badge, Checkbox, Select, Switch, Tag, Text, Heading, Textarea, IconButton, Container.

Molecules (`src/molecules/`): atoms combined to do one small, complete job — Card, Callout, SectionHeading, Tooltip, Stats.

Organisms (`src/organisms/`): distinct, complex regions built from molecules and atoms — NavBar, Footer.

App sections (`apps/web/src/sections/`): compose organisms, molecules, and atoms into the actual regions of a page — Hero, Features, Pricing, and so on. These live in the consuming app, not the library.

For junior devs

Before building anything new, walk one tier down. Need a new molecule? Check whether the atoms it needs already exist. Need a new atom? Check whether Box already gives you the layout you need and you're really just adding variant styling. Almost everything should be composition, not invention.

The file shape every component should follow

Every primitive, atom, molecule, and organism should live in its own directory with a fixed file set:

```
Button/
├─ Button.tsx          # public prop API + logic
├─ Button.styles.ts    # styled-components definitions
├─ Button.test.tsx     # unit test + accessibility test
├─ Button.stories.tsx  # Storybook CSF3 stories
└─ index.ts           # barrel re-export
```

When building something new, find the closest existing analog and copy its shape rather than inventing a new pattern. Container is the simplest real example of pure composition — Box with three lines of extra CSS:

```
// atoms/Container/Container.tsx
export type ContainerProps = BoxProps;
export function Container(props: ContainerProps) {
  return <StyledContainer {...props} />;
}
```

```
// atoms/Container/Container.styles.ts
export const StyledContainer = styled(Box)`
  max-width: var(--container-max);
  margin-left: auto;
  margin-right: auto;
  padding-left: var(--container-pad);
  padding-right: var(--container-pad);`
```

`;
`;

Every public export — every component and its prop/variant types — should be re-exported from one top-level `src/index.ts`, grouped by tier under comment headers. Nothing outside the package should be importable unless it appears in that one file.

If a component is added but never re-exported from `src/index.ts`, it will build fine, test fine, and show up in Storybook — but no other package can import it. This is the single most common “why isn’t my new component available in the app” mistake.

Chapter 5 --- Styling: styled-components and Transient Props

The problem transient props solve

styled-components lets you attach arbitrary props to a styled element and read them inside its CSS template literal to branch styling logic — but by default, any prop you pass also gets forwarded to the underlying DOM node. Pass `variant="primary"` to a `styled.button`, and React dutifully renders `<button variant="primary">`, which is not a real HTML attribute and produces a console warning (and invalid markup).

The fix styled-components provides is the **\$ prefix**: any prop name starting with `$` is treated as “transient” — available inside the styled component’s CSS, but automatically stripped before anything reaches the DOM.

The pattern worth using consistently

Every styled component should follow the same split: the public-facing `.tsx` file exposes a clean, semantic prop API (`variant`, `size`, plain booleans), and translates each prop into its `$`-prefixed transient equivalent right before handing off to the styled component:

```
// atoms/Button/Button.tsx
export function Button({
  variant = 'primary', size = 'md', type = 'button', ...props
}: ButtonProps) {
  const [spacing, rest] = extractSpacingProps(props);
  return (
    <StyledButton $variant={variant} $size={size} $spacing={spacing}
      type={type} {...rest} />
  );
}

// atoms/Button/Button.styles.ts
export const StyledButton = styled.button<{
  $variant: ButtonVariant; $size: ButtonSize; $spacing: SpacingProps;
}>`
  /* shared base styles */
  ${({ $size }) => sizeStyles[$size]}
  ${({ $variant }) => variantStyles[$variant]}
  ${spacingCss}
`;
```

`sizeStyles` and `variantStyles` should be lookup tables typed as `Record<ButtonSize, css...>` / `Record<ButtonVariant, css...>` — so adding a new size or variant is a matter of adding one new key to a table, not writing new conditional logic.

A shared spacing-prop mixin

One more pattern worth threading through nearly every component: a `SpacingProps` type that lets any component accept layout props like `p`, `px`, `mt`, typed against the `tokens` package’s `SpaceKey` union so only real spacing-token values type-check. An `extractSpacingProps` helper strips those

props out of what reaches the DOM, and a shared `spacingCss` template literal turns whichever spacing props were passed into real CSS declarations.

For senior devs

Typing `SpacingProps` values as template-literal types derived directly from the `tokens` package's `SpaceKey` union (``space-${SpaceKey}``) means adding a new spacing token automatically makes it a valid value everywhere `p/mt/px` etc. are accepted, with no separate type definition to update.

Chapter 6 --- Light and Dark Mode

The mechanism worth copying

Because every color a component uses is a CSS custom property (`var(--color-accent)`), never a hardcoded value or a JS token lookup), theming reduces to a CSS problem, not a React problem. The colors token file should write the light palette to `:root` and the *same variable names*, under dark values, into a `[data-theme='dark'] { ... }` block. Because CSS custom properties cascade down through the DOM, any element inside a subtree with `data-theme="dark"` on an ancestor picks up the dark values automatically — no component re-render required.

ThemeProvider should wire this to a mode prop:

```
export function ThemeProvider({ children, mode = 'light' }) {
  return (
    <StyledThemeProvider theme={theme}>
      <GlobalStyle />
      <div data-theme={mode === 'dark' ? 'dark' : undefined}>
        {children}
      </div>
    </StyledThemeProvider>
  );
}
```

A `useThemeMode` hook should own the actual mode state: seed from `localStorage`, fall back to the `prefers-color-scheme` media query if nothing is stored, persist on every change, and expose `{ mode, setMode, toggleMode }`. The app's root layout calls that hook and feeds mode into `ThemeProvider`, and passes `toggleMode` down to a theme toggle button.

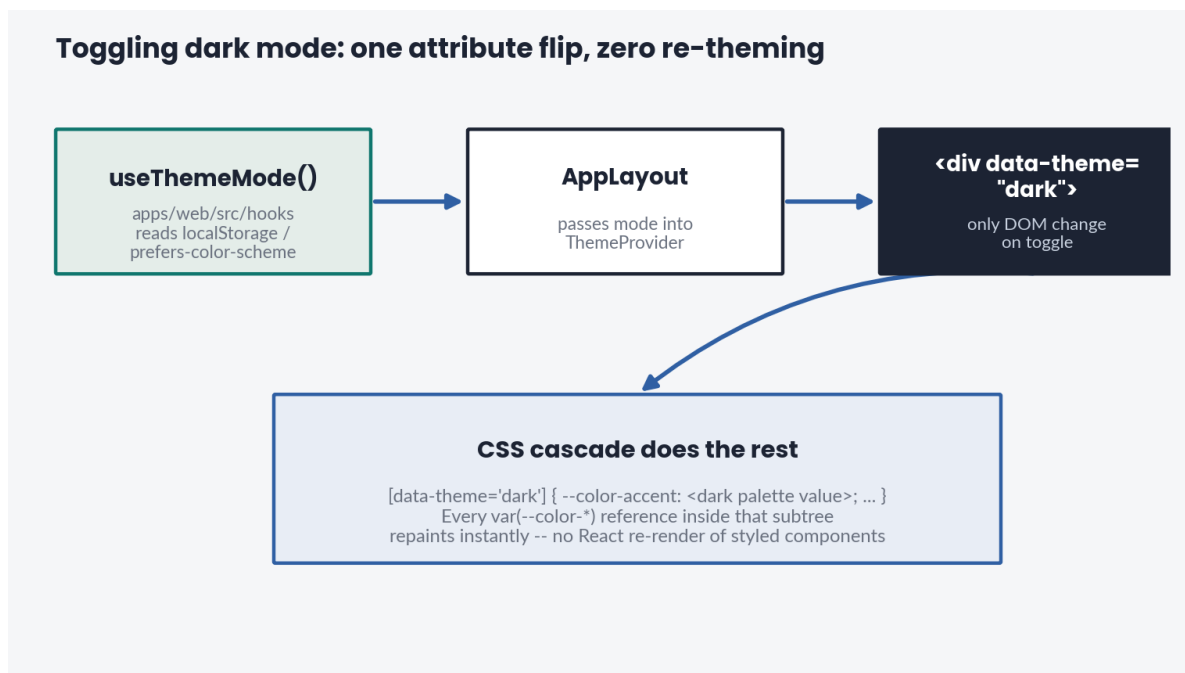


Figure 5: Toggling dark mode is one DOM attribute flip; the CSS cascade repaints everything

Only color tokens need a dark variant. Spacing, typography, and semantic (component-anatomy) tokens can stay identical in both modes — dark mode in this system is purely a repaint, never a re-layout.

Storybook should mirror the exact same mechanism for previewing components in isolation: a theme toolbar global, with a decorator that passes its value into ThemeProvider around every story.

Chapter 7 --- Implementing the UI Kit on the Frontend

Consuming the design system in a real app

The frontend app should contain almost no styling of its own — every visual decision is delegated to the `ui` package. The page composition should be intentionally linear and easy to reorder: a root `App.tsx` renders a layout component wrapping an ordered list of section components imported from a `sections/` directory. Reordering the page means reordering that list, not touching the sections themselves. Each section should be a standalone component built purely from `ui` primitives, atoms, and molecules — no bespoke CSS living in the app.

A root layout component is the one place app-level concerns should live: it calls `useThemeMode()`, wraps everything in `ThemeProvider`, and renders shared chrome like a nav bar and footer around the page content.

Separating API requests from presentational components

This is the single most important frontend architectural decision on top of the component library, and it's worth getting right from the first feature. A **presentational component** should only ever receive data through props and render it — it should never import a fetch call, an API client, or a data-fetching hook directly. A **container** (either a wrapper component or, more commonly today, a custom hook) owns the data: it fetches, caches, and manages loading/error state, then hands plain props down to the presentational component underneath.

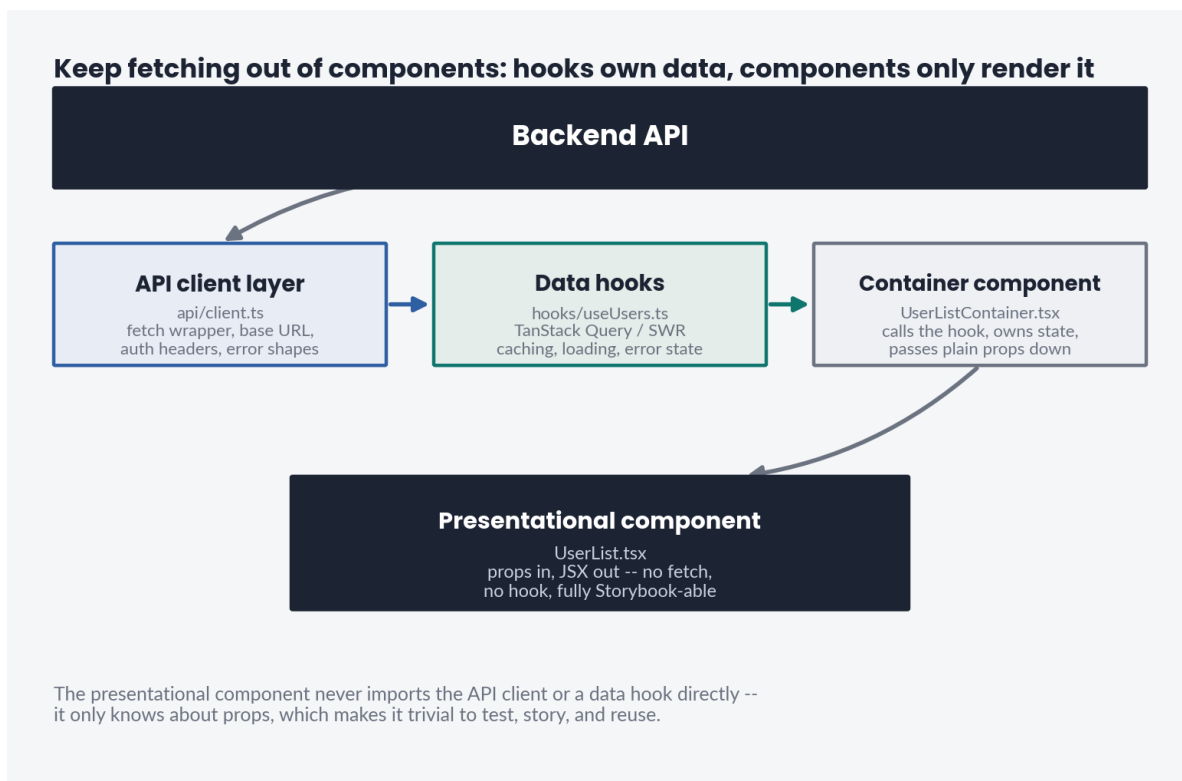


Figure 6: Keep fetching out of components: hooks own data, components only render it

The pattern breaks into three layers:

An API client layer (`api/client.ts`) is a thin wrapper around `fetch` (or `axios`) that owns the base URL, default headers, auth token attachment, and a consistent error shape. Nothing above this layer should ever call `fetch` directly — every request goes through this one module, so changing how auth headers are attached, or how errors are parsed, happens in exactly one place.

Data hooks (`hooks/useUsers.ts`) wrap the API client with a data-fetching library — **TanStack Query** or **SWR** are the two dominant choices in 2026 — which gives you caching, request de-duplication, background refetching, and loading/error state for free, instead of hand-rolling `useState/useEffect` data fetching in every component that needs data. A hook like `useUsers()` returns `{ data, isLoading, error }` and nothing else; it has no opinion about how that data is rendered.

Container components (or the hook consumed directly inside a route/page component) call the data hook, own any local UI state (like which tab is active or which row is selected), and pass plain, serializable props down to a presentational component.

Presentational components (`UserList.tsx`) take props in and return JSX — no `fetch`, no data hook, no knowledge that an API even exists. This is what makes them trivially reusable, testable in isolation, and directly storyable in Storybook with hard-coded mock data, since they don't need a live backend or a mocked network layer to render.

For junior devs

A quick test for whether a component has stayed presentational: could you render it in Storybook with made-up data and zero network mocking? If the answer is no, a data hook or API call has leaked into a component that should only know about props.

For senior devs

Where you draw the container/presentational line matters less than being consistent about it. Some teams put the data hook call directly in the route/page component and skip a separate “container” file entirely — that's fine, and arguably the more common pattern since hooks replaced the class-based container pattern. What matters is that the *leaf* components rendering lists, cards, and forms never import a data hook themselves.

A folder structure worth starting from

```
apps/web/src/
├── api/
│   └── client.ts           # the only place fetch()/axios lives
├── hooks/
│   ├── useUsers.ts        # data hooks, one per resource
│   └── useThemeMode.ts
├── components/
│   └── UserList.tsx        # presentational, no data awareness
├── sections/
│   └── Hero.tsx            # composed from ui + hooks
├── layouts/
│   └── AppLayout.tsx
└── App.tsx
```

Environment-specific API base URLs, timeouts, and retry policy belong in the API client layer, configured once from environment variables — never scattered across individual fetch calls in components. When the backend URL changes between local, staging, and production, exactly one file should need to change.

Chapter 8 --- Backend Fundamentals

Prisma: a type-safe data layer

Prisma is a TypeScript ORM that lets you define your database schema declaratively in a `schema.prisma` file, then generates a fully type-safe client from it — every query is checked against your actual schema at compile time, not discovered as a runtime error. It bundles three tools: the schema itself, **Prisma Client** for type-safe queries, and **Prisma Migrate** for versioning schema changes alongside your application code.

A few conventions are worth adopting from day one. Use PascalCase, singular model names (User, not users), and camelCase field names. Always define both sides of a relation so the schema stays self-documenting. Index any field used in a where, orderBy, or relation lookup — an unindexed foreign key is one of the most common causes of a slow API that “worked fine in development” with ten rows and fell over in production with a million.

Create exactly **one** global `PrismaClient` instance and reuse it throughout the app. Instantiating a new `PrismaClient` per request (a common mistake inside serverless functions or hot-reloading dev servers) opens a new connection pool each time and will exhaust your database’s connection limit under real load.

For migrations: `prisma migrate dev` in development generates and applies a migration from schema changes; `prisma migrate deploy` applies already-generated, checked-in migrations in CI and production — it never generates new ones, which is exactly the safety property you want in a deploy step. `prisma db push` skips migration files entirely and should stay a prototyping-only tool, never a production deployment path, since it leaves no history of what changed or why.

Rate limiting

Rate limiting exists to protect an API from being overwhelmed — whether by a genuine traffic spike, a buggy client stuck in a retry loop, or a deliberate abuse/brute-force attempt — by rejecting requests once a client exceeds a defined budget. Three algorithms cover most real needs:

Fixed window counts requests in a fixed interval (say, 100 requests per 60-second window) and resets the counter at each boundary. It’s simple to implement and reason about, but it allows a burst of up to double the limit right at the window boundary (100 requests in the last second of one window, then another 100 in the first second of the next).

Sliding window tracks requests over a continuously moving time frame rather than a fixed boundary, which smooths out that boundary-burst problem at the cost of a bit more bookkeeping.

Token bucket gives each client a bucket that refills at a steady rate and is drained by each request; if the bucket is empty, the request is rejected. It’s the strongest general-purpose default for APIs because it naturally absorbs short bursts (a client can spend saved-up tokens quickly) while still enforcing a steady average rate over time.

Avoid

Don’t rate-limit only by IP address. Shared IPs (corporate NAT, mobile carriers) will collectively hit the limit and lock out unrelated users, while a distributed attacker rotating IPs sails right

through. Rate limit by authenticated user or API key wherever a request is authenticated, and reserve IP-based limits for unauthenticated endpoints like login and signup.

Auth guards

An **auth guard** is middleware that runs before a route handler's actual logic and answers two separate questions: **authentication** ("who is making this request?") and **authorization** ("is this specific requester allowed to do this specific thing?"). Keeping both checks in a guard that runs *before* the handler — rather than scattered as `if` statements inside business logic — means a route can never accidentally skip an auth check, and the check is written and tested in exactly one place.

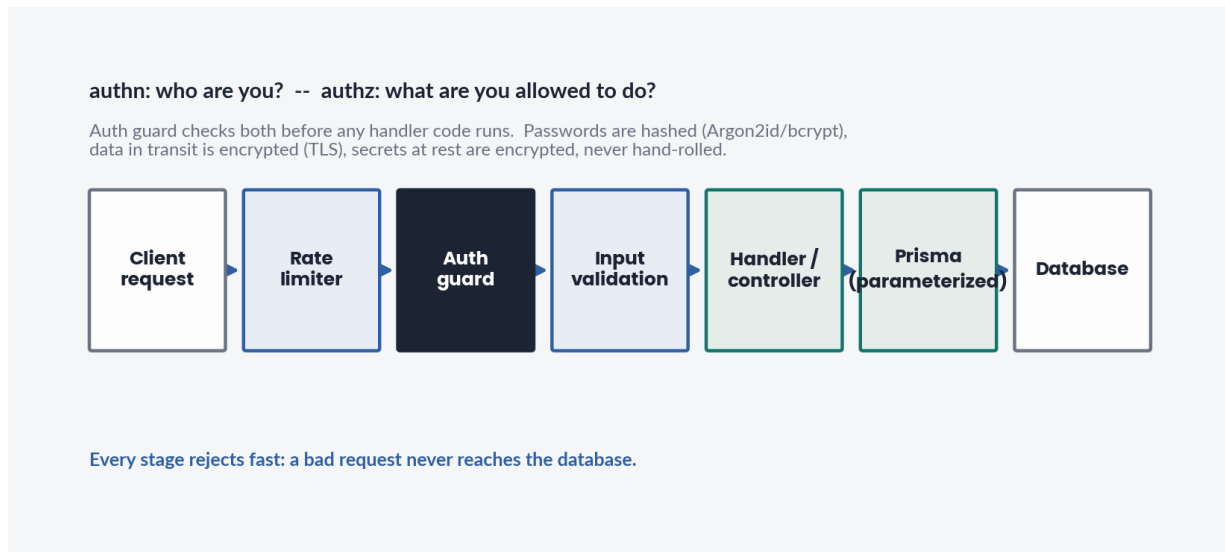


Figure 7: A request should be rejected as early as possible in the pipeline

A guard typically does three things in order: verify the credential presented (a session cookie, a bearer token, an API key) and resolve it to an identity; load whatever authorization data that identity needs (roles, permissions, ownership of the resource being requested); and reject the request with a 401 (not authenticated) or 403 (authenticated, but not allowed) before any handler code runs.

401 Unauthorized means "I don't know who you are." 403 Forbidden means "I know exactly who you are, and the answer is no." Conflating the two in API responses makes debugging auth issues far harder than it needs to be — and for genuinely sensitive resources, returning 404 instead of 403 (so an unauthorized user can't even confirm a resource exists) is a deliberate, defensible choice.

Authentication and authorization: which to use when

Session-based auth stores auth state server-side (in memory, Redis, or a database), with the client holding only an opaque session ID in an `HttpOnly` cookie. It's the simplest secure choice for a traditional server-rendered web app with a single backend, and sessions can be revoked instantly by deleting server-side state — a real advantage when immediate logout or ban enforcement matters. It struggles once you have multiple backend services that don't share session storage.

JWTs (JSON Web Tokens) are signed tokens the server issues and the client stores and resends; the token itself carries the claims, so no server-side lookup is needed per request. That statelessness is exactly what makes JWTs the right fit for REST APIs, mobile clients, and microservice-style backends that need to scale horizontally without a shared session store. The tradeoff is real: a JWT can't be revoked before it expires without extra infrastructure (a blocklist, short expiries plus refresh tokens), so keep access-token lifetimes short (5–15 minutes) and pair them with a longer-lived refresh token stored in an `HttpOnly` cookie.

API keys are a long random string sent with every request, checked against a stored value. They're the right tool for server-to-server integrations, internal tooling, and usage metering — not for human login. They should be treated as secrets: never logged, never committed, rotatable without a full redeploy, and scoped to only the permissions a given integration actually needs.

OAuth 2.0 / OIDC is the standard when a user needs to grant *your* app access to *their* data on another platform (social login, “Sign in with Google,” enterprise SSO). It's a delegation protocol, not primarily a login mechanism, though OpenID Connect (OIDC) layers identity on top of it to make it work for login too. Don't hand-roll an OAuth flow — use a maintained library or an identity provider (Auth0, Clerk, Cognito, an internal IdP) rather than implementing the token exchange and PKCE flow from scratch.

A common, sound production pattern: use OAuth 2.0 / OIDC for the actual login (letting a trusted identity provider handle credentials), then issue your own short-lived JWTs for subsequent API access. You get delegated, audited login plus stateless, scalable API auth, without reinventing either.

Encryption and hashing

These two are frequently confused and used interchangeably, but they solve different problems and are not interchangeable in practice.

Hashing is one-way. A hash function turns input into a fixed-size output that cannot be reversed back to the original input. This is exactly what you want for passwords: the server should never be able to recover a user's actual password, only verify that a freshly hashed login attempt matches the stored hash. Passwords should always be hashed with a purpose-built, slow, memory-hard algorithm — **Argon2id** is the current best default, with **bcrypt** a well-tested, still-viable fallback where Argon2 isn't available. Never use a fast general-purpose hash like plain SHA-256 or MD5 for passwords — their speed, which is a virtue for file integrity checks, is exactly what makes them crackable at scale via brute force. Always use a unique, random salt per password (any modern hashing library does this for you automatically) — never a shared global salt, and never a salt derived from the username.

Encryption is two-way and reversible. Data is encrypted with a key and can be decrypted back to its original form with that same key (symmetric, e.g. AES) or a paired key (asymmetric, e.g. RSA/ECC). Use encryption for anything the system legitimately needs to read back later — API secrets, connection strings, personally identifiable information at rest, data in transit. Encryption in transit means **TLS everywhere**, including internal service-to-service calls, not just the public-facing edge. Encryption at rest means secrets and sensitive columns are encrypted in the database or a dedicated secrets manager (AWS KMS, HashiCorp Vault, or your cloud provider's equivalent) — not sitting in plaintext environment variables checked into a repository.

Avoid

Never write your own cryptographic primitives, and never store passwords with encryption instead of hashing — encryption implies a key exists that can decrypt them, which means anyone with that key (including an attacker who compromises it) can recover every plaintext password at once. Use established, audited libraries for both hashing and encryption; this is not the place to be clever.

What to avoid for a fast, secure API

The patterns below account for the large majority of real-world API incidents and slowdowns, and map closely onto the [OWASP API Security Top 10](#).

Avoid

Broken object-level authorization. Checking that a user is *logged in* is not the same as checking that they own the *specific record* they're requesting. GET /orders/1234 must verify the requester owns order 1234, not just that they're authenticated — this single class of bug is consistently the most common API vulnerability found in the wild.

N+1 queries. Fetching a list, then querying the database again inside a loop for each item's related data, turns one page load into hundreds of round trips. Use include/select (Prisma's relation-loading) or a batching layer (DataLoader) to fetch related data in one query instead.

Unbounded resource consumption. An endpoint with no rate limit, no pagination cap, and no request-size limit is an invitation for a slow client, a buggy retry loop, or a deliberate attacker to degrade the service for everyone else. Always paginate list endpoints, cap page sizes, and enforce request body size limits.

Trusting client input. Validate and sanitize every field of every request server-side, even ones a well-behaved frontend would never send incorrectly — the frontend is not a security boundary, and anyone can call your API directly with a tool like curl or Postman.

Verbose error responses. A stack trace, an internal file path, or a raw database error returned to the client hands an attacker a map of your internals. Log the detailed error server-side; return a generic, safe message to the client.

Overly permissive CORS. Access-Control-Allow-Origin: * on any endpoint that reads authenticated data lets any website make credentialed requests on a logged-in user's behalf. Scope CORS to the exact origins that need it.

Blocking the event loop. In a Node.js API, a synchronous CPU-heavy operation (large in-process image processing, uncontrolled regex, a huge synchronous JSON parse) blocks every other request being served by that process. Offload heavy work to a background job queue.

Chapter 9 --- Quality Tooling

ESLint: one shared flat config

A shared `packages/eslint-config` should export a single **flat config** (the modern ESLint config format — an array of config objects rather than the older `.eslintrc` cascading format), built with `typescript-eslint`'s `tseslint.config(...)` helper. Every app and package should have its own one-line `eslint.config.mjs` that imports and re-exports this shared config, so there is exactly one place linting rules are defined.

A sensible layering: ignore build output → ESLint's own recommended rules → `typescript-eslint`'s **strict** and **stylistic** rule sets (a stricter bar than just "recommended") → framework-specific plugin rules (React, `react-hooks`, `jsx-ally` if it's a React project) → a couple of house rules → `eslint-config-prettier` last, which turns off any stylistic rule that would conflict with Prettier's formatting. Run CI with `--max-warnings 0`, so warnings fail the build exactly like errors.

TypeScript: a small set of shared configs

A `packages/tsconfig` package exporting a handful of base `tsconfig.json` files — and which one a package extends tells you what kind of package it is — keeps configuration consistent without copy-pasting compiler options everywhere:

- **base.json** — the shared root: modern target, strict mode fully on, `noUnusedLocals/noUnusedParameters`. Never used standalone.
- **node-library.json** — extends base for CommonJS Node output. Used by a plain Node API service.
- **react-app.json** — extends base with DOM libs and `noEmit: true` (the bundler handles compilation). Used by a Vite/Next.js frontend.
- **react-library.json** — extends base with DOM libs plus `declaration/declarationMap: true`, since this package ships `.d.ts` files for consumers. Used by the UI package.

A plain-TypeScript library with no React or JSX in it at all (like the `tokens` package) should extend `base.json` directly rather than `react-library.json`.

Testing: pick per package, not per project

It's reasonable — and increasingly common — to use **Vitest** for the library packages (UI, `tokens`) and **Jest** for the applications, rather than forcing one test runner everywhere. Vitest is Vite-native and shares a Vite-based package's module graph, making it dramatically faster in watch mode with effectively no extra config. Jest remains a battle-tested choice for full applications with more complex test-environment needs, and its API is close enough to Vitest's (`describe/it/expect, vi.fn()` vs `jest.fn()`) that moving between codebases doesn't require relearning a testing mental model.

Whichever runner is used for the UI package, wire in **jest-axe** (or the Vitest equivalent) for automated accessibility checks, and make an explicit assertion part of every component test:

```
expect(await axe(container)).toHaveNoViolations();
```

For junior devs

That one line is not boilerplate to skip past — it's a real automated accessibility check (contrast, ARIA attributes, semantic markup) run on every component, every test run. If a new component's test fails on that line, treat it exactly as seriously as a failing assertion on the component's actual behavior.

Chapter 10 --- Storybook & Component-Driven Development

Why build components in isolation

Component-Driven Development is the practice of building and verifying a UI component on its own, outside the full application, before wiring it into a real page. A component built in isolation is forced to declare all of its own dependencies (props, not implicit page context), which tends to produce more reusable, more testable components — and it gives designers and product stakeholders something concrete to review long before a feature is fully assembled.

Storybook is the tool that makes this practical: a standalone dev server that renders one component at a time against different prop combinations (“stories”), with live controls to tweak props in the browser. Modern Storybook stories are written in **CSF3** (Component Story Format 3) — each file default-exports metadata about the component, and named-exports one object per state to showcase:

```
// atoms/Button/Button.stories.tsx (shape, not exact code)
export default {
  component: Button,
  argTypes: { variant: { control: 'select' }, size: { control: 'select' } },
};

export const Primary = { args: { variant: 'primary' } };
export const Disabled = { args: { disabled: true } };
```

A Storybook setup worth adopting

It’s worth running **separate Storybook instances**, one per package that has UI to show, each on its own port so more than one can run at once — one for the component library, one for the frontend app (useful for whole-page reference stories, not just isolated components), and one for the tokens package itself, rendering raw token swatches with `@storybook/html` rather than components.

For designers

A tokens-package Storybook that renders every primitive and semantic token as a literal swatch — light and dark side by side, with no component involved — is the single fastest way for a designer to see the entire palette in one place and check a hex value against what’s actually shipped, rather than trusting a design file might be in sync with code.

Chapter 11 --- Continuous Integration

A CI workflow should run one job on every push and pull request targeting the main branch, with a concurrency group that cancels superseded runs so an outdated push doesn't waste CI time. A proven, linear step order:

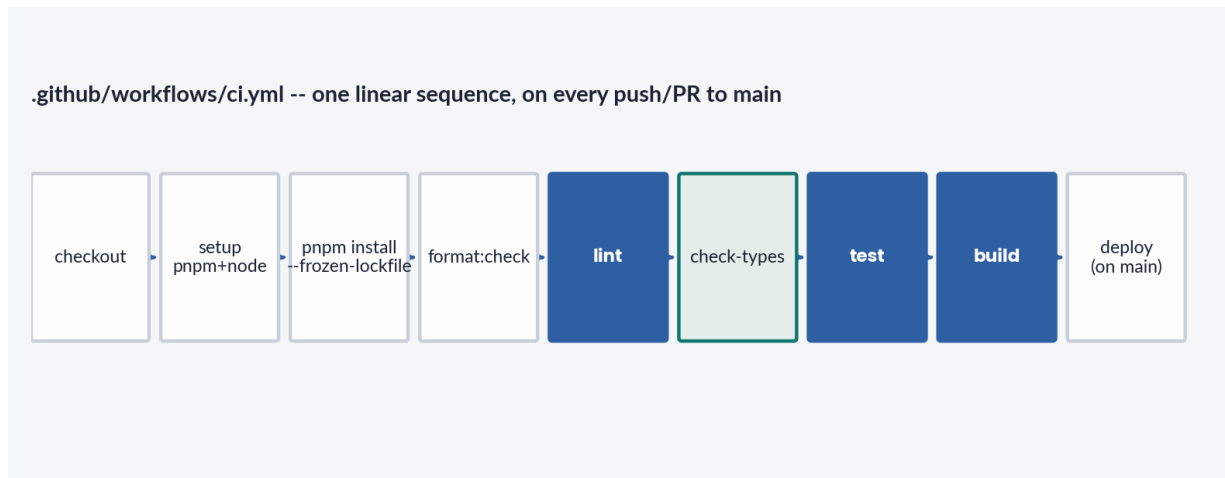


Figure 8: CI should run the exact same quality gates you run locally, in the same order, before every push

1. Check out the repository.
2. Set up pnpm and Node, with pnpm's store cached.
3. `pnpm install --frozen-lockfile` — fail the build outright if the lockfile is out of sync with any `package.json`, rather than silently updating it.
4. Restore a cache of the `.turbo` directory, keyed on the commit SHA.
5. `pnpm format:check` — formatting, checking only.
6. `pnpm lint` — across every package.
7. `pnpm check-types` — across every package.
8. `pnpm test` — across every package.
9. `pnpm build` — across every package.
10. Deploy, gated to the main branch only, after every prior step passes.

Match this order locally before pushing: `format check` → `lint` → `type-check` → `test` → `build`. Matching CI's order locally means a broken push is caught on your own machine first, faster than waiting on a CI run.

Chapter 12 --- Cheat Sheet

Commands

<code>pnpm install</code>	<code># install everything</code>
<code>pnpm dev</code>	<code># run dev servers</code>
<code>pnpm build / lint / check-types / test</code>	<code># repo-wide, dependency-ordered</code>
<code>pnpm format:check</code>	<code># formatting check (not turbo'd)</code>
<code>pnpm storybook</code>	<code># all Storybooks that exist</code>
<code>pnpm --filter @yourorg/ui test</code>	<code># scope any task to one package</code>
<code>pnpm --filter @yourorg/ui storybook</code>	<code># ui's Storybook only</code>

Where things should live

- Design token **values** → `packages/tokens/src/defaults/*.ts`
- Design token **derivation logic** → `packages/tokens/src/ createSemanticTokens.ts`
- Token → CSS **serialization** → `packages/ui/src/theme/ tokens/*.ts`
- **Components** → `packages/ui/src/ – primitives/, atoms/, molecules/, organisms/`
- **Public exports** → `packages/ui/src/index.ts` (nothing is importable elsewhere without this)
- **API client** → `apps/web/src/api/client.ts` (the only place `fetch/axios` lives)
- **Data hooks** → `apps/web/src/hooks/`
- **Page composition** → `apps/web/src/App.tsx` and `apps/web/src/sections/`
- **Auth guards / middleware** → `apps/api/src/middleware/`
- **Database schema** → `apps/api/prisma/schema.prisma`
- **Shared lint rules** → `packages/eslint-config/index.js`
- **Shared tsconfig bases** → `packages/tsconfig/*.json`
- **CI definition** → `.github/workflows/ci.yml`

Chapter 13 --- Where to Start, By Role

If you're a junior developer

Start by running `pnpm storybook` and clicking through every component in the UI package. Then open a component like `Button` end to end — all five files — and read it as a template. Early tasks will almost always be one of: adding a new variant to an existing component (edit the lookup table in `X.styles.ts`), building a new molecule from existing atoms, wiring an existing component into a new spot in a frontend page, or adding a new data hook that follows the existing API-client pattern. Before writing new CSS, check whether `Box` and the shared `SpacingProps` mixin already give you what you need. Before writing a new fetch call, check the API client layer already exists for that purpose.

If you're a senior developer

The seams worth getting right before a team scales up around them: the `Turborepo` task graph and its `^build` dependencies (Chapter 2), the `mergeTokens` escape hatch for future rebranding (Chapter 3), the container/presentational boundary on the frontend (Chapter 7), and the `auth-guard-before-handler` discipline on the backend (Chapter 8) — all four are real architectural decisions that are far cheaper to get right on day one than to retrofit once a dozen engineers are shipping around them.

If you're a product designer

Run `pnpm storybook` and start with the `tokens` package's Storybook — the color, spacing, and typography stories are the ground truth for the palette and scale, more reliable than any static reference doc. When handing off a design, reference semantic token names (`color-accent`, `space-6`) rather than raw hex codes or pixel values — it lets engineers implement it as a token reference rather than a magic number, which is what keeps the system consistent as it grows. If a design calls for something that doesn't map to an existing atom or molecule, that's a useful signal worth raising directly with engineering — it may mean either an existing component needs a new variant, or the design itself should reconsider using what's already available.

Appendix --- Further Reading

This guide covers the pattern in general terms; for deeper background on each underlying concept:

- Turborepo's own documentation on structuring a repository: <https://turborepo.dev/docs/crafting-your-repository/structuring-a-repository>
- Brad Frost's original Atomic Design methodology: <https://atomicdesign.bradfrost.com/chapter-2/>
- Storybook's Component Story Format reference: <https://storybook.js.org/docs/api/csf>
- styled-components' own docs on transient props: <https://styled-components.com/docs/api>
- Prisma's documentation on schema, migrations, and best practices: <https://www.prisma.io/docs/orm/more/best-practices>
- The OWASP API Security Top 10 (2023): <https://owasp.org/API-Security/editions/2023/en/Ox11-t10/>
- A comparison of password hashing algorithms (Argon2 vs bcrypt vs scrypt): <https://stytch.com/blog/argon2-vs-bcrypt-vs-scrypt/>
- Rate limiting algorithms compared (token bucket vs sliding/fixed window): <https://blog.arcjet.com/rate-limiting-algorithms-token-bucket-vs-sliding-window-vs-fixed-window/>